



Report writing

Artificial Intelligence

Assignment # 1

N Queen Puzzle

2023-01-13

Contents

Introduction	3
N-Queen BFS and DFS	3
BFS(Brath first search)	3
DFS(depth-first search)	3
BFS vs DFS	4
N-queens time complexity	4
Example of BFS, DFS and Backtracking	5
N-Queen Backtracking and Genetic Algorithm	5
Backtracking	5
Genetic Algorithm	5
Backtracking vs GA	6
Example of Backtracking and Genetic Algorithm	7
Conclusion	7
References	7

Introduction

The N-Queens problem is a classic problem of placing N queens on an NxN chessboard in such a way that no two queens are attacking each other. A queen can attack horizontally, vertically, and diagonally. This problem can be solved using different techniques such as backtracking, recursive, genetic algorithm, and branch and bound. These methods all have their advantages and disadvantages and they differ in terms of time complexity, space complexity, and the quality of solutions. The 8-Queens problem is a well-known problem in computer science and has been widely studied and applied in various fields such as Artificial Intelligence, Combinatorial Optimization, and Game Design[1].

This report uses the BFS, DFS, Backtracking, and GA to solve the N-queen problem. Additionally, one variant of this problem is the 8-queens problem, which is the problem of placing 8 queens on an 8x8 chessboard such that no two queens are attacking each other. This problem has 92 distinct and 12 unique solutions.

N-Queen BFS and DFS

BFS(Brath first search)

It is possible to solve the N-Queens problem using a Breadth-First Search (BFS) algorithm, although it is not a common approach. The basic idea is to use a queue to store the partial solutions (i.e., the positions of the queens on the chessboard) and expand them by adding new queens in the next empty rows[2].

The algorithm starts by placing the first queen in the first row and first column and adding it to the queue. Then, it takes the next partial solution from the queue and checks if it is a complete solution (i.e., all rows have a queen). If it is, it is added to the list of solutions. If it is not, the algorithm generates all the new partial solutions by adding a queen to the next empty rows and columns and checks if it is a safe move (i.e., no clashes with the queens already placed). If it is, it is added to the queue.

The algorithm repeats this process until the queue is empty. In the end, it returns the list of solutions.

The time complexity of this BFS approach is $O(N! * N^2)$ where N is the size of a chessboard. The space complexity is $O(N!)$. It is not a good approach to solve the N-queens problem as the time complexity is high and the space complexity is huge.

DFS(depth-first search)

The 8-Queens problem can be solved using a Depth-First Search (DFS) algorithm. The basic idea is to use recursion to generate all the possible configurations of queens on the chessboard, and use backtracking to discard the invalid ones.

The algorithm starts by placing a queen in the first column of the first row and recursively calls itself to place the next queen in the next row. For each recursive call, it checks if it is a safe move (i.e., no clashes with the queens already placed) and, if it is, it adds the queen to the current configuration. If it reaches a point where it cannot place the next queen in a safe position, it backtracks and removes the last queen placed from the current configuration.

The algorithm repeats this process until it reaches a complete solution (i.e., all 8 queens placed on the chessboard without any clashes) or all the possibilities have been exhausted.

The time complexity of this DFS approach is $O(N! * N^2)$ where N is the size of a chessboard. The space complexity is $O(N)$. It is not a good approach to solve the 8-queens problem as the time complexity is high and the space complexity is huge.

It is important to note that other methods can solve the 8-queens problem more efficiently such as backtracking and genetic algorithm

BFS vs DFS

Both Breadth-First Search (BFS) and Depth-First Search (DFS) can be used to solve the 8-Queens problem, but they have some key differences[3].

BFS uses a queue data structure to store the partial solutions and generates new solutions by adding queens to the next empty rows and columns. It is a systematic approach that generates all the solutions in a breadth-first manner, which means it explores all the solutions at the same depth level before moving on to the next level. The time complexity of BFS is $O(N! * N^2)$ where N is the size of a chessboard. The space complexity is $O(N!)$ [4].

DFS uses recursion and backtracking to generate all the possible configurations of queens on the chessboard. It is a recursive approach that generates all the solutions in a depth-first manner, which means it explores as far as possible along each branch before backtracking. The time complexity of DFS is $O(N! * N^2)$ where N is the size of a chessboard. The space complexity is $O(N)$.

In terms of solving the 8-queens problem, DFS and BFS are not efficient as the time complexity is high and the space complexity is huge.

N-queens time complexity

One popular approach to solving this problem is to use backtracking. The basic idea is to place queens one by one in different columns, starting from the leftmost column. When we place a queen in a column, we check for clashes with already placed queens. In case of a clash, we remove the queen and backtrack. If we do not find a safe position for a queen in the current column, we return false

The time complexity for the N-queens problem is $O(N!)$ in the worst case. As the size of the problem increases, it becomes impractical to solve it using backtracking. Space Complexity is $O(n)$ [5].

Example of BFS, DFS and Backtracking

Here is an example of the git-hub code [6] time complexity of BFS, DFS, and Backtracking of 8-queen problem:

```
Size of board: 8
Total time taken for DFS solutions:      58.957321
Total time taken for BFS solutions:      305.363011
Total time taken for Backtrack solutions: 0.028914
```

The out shows that Depth-First Search (DFS) is considered to be more space-efficient than Breadth-First Search (BFS). And backtracking is considered to be more efficient than DFS because it's a systematic and deterministic algorithm that incrementally builds up a solution and uses backtracking to discard invalid solutions.

N-Queen Backtracking and Genetic Algorithm

Backtracking

The 8-Queens problem can be solved using the backtracking algorithm. Backtracking is a general algorithmic technique that involves incrementally building up a solution to a problem, and then undoing (or "backtracking" on) the last step if it leads to an invalid solution.

The basic idea of the backtracking algorithm for the 8-Queens problem is to place queens one by one in different columns, starting from the leftmost column. For each column, we check all the rows to find a safe position to place the queen. We keep track of the positions of the queens on the chessboard using a 2D array or a 1D array of size N.

When we place a queen in a column, we check for clashes with already placed queens. In case of a clash, we remove the queen and backtrack to the previous state. If we find a safe position for a queen in the current column, we proceed to the next column. If we do not find a safe position for a queen in the current column, we return false and backtrack to the previous state.

When we reach the last column, it means we have found a solution and we can store it. We proceed to find the next solution by backtracking to the previous state and changing the position of the queen in the current column.

The time complexity of the backtracking algorithm is $O(N! * N^2)$ where N is the size of the chessboard and the space complexity is $O(N^2)$. Backtracking is a very efficient algorithm for solving the 8-queens problem, as it reduces the number of possibilities to explore.

Genetic Algorithm

The 8-Queens problem can be solved using a genetic algorithm (GA) approach. This approach uses a population of candidate solutions, represented by chromosomes, that evolve through a process of selection, crossover (recombination), and mutation [7].

The initial population is generated randomly and each individual is assigned a score based on the fitness function, which evaluates the number of attacking queens. The individuals with the highest fitness score are selected for the next generation.

Crossover is the process of recombining the chromosomes of two individuals to create new offspring. This allows for the exchange of genetic information between individuals and can lead to new solutions.

The mutation is the process of randomly altering the value of one or more genes in the chromosome. This can introduce new genetic diversity and can also lead to new solutions.

The process is repeated for several generations and the best individual from the final generation is considered the solution to the 8-Queens problem. It's important to note that GA is a probabilistic method and the solution may not be the optimal one. Additionally, the algorithm's performance depends on the parameters such as population size, mutation rate, and selection criteria.

The time complexity of a genetic algorithm is $O(NGT)$ where N is the size of the population, G is the number of generations and T is the number of evaluations (fitness function calculations) per individual. The space complexity is $O(N)$.

Backtracking vs GA

Backtracking and Genetic Algorithm (GA) are both methods that can be used to solve the 8-Queens problem, but they have some key differences [8].

Backtracking is a systematic and deterministic algorithm that incrementally builds up a solution to a problem, and then undoes (or "backtracking" on) the last step if it leads to an invalid solution. It starts by placing queens one by one in different columns, starting from the leftmost column, checking all the rows to find a safe position to place the queen. The time complexity of the backtracking algorithm is $O(N! * N^2)$ where N is the size of the chessboard and the space complexity is $O(N^2)$.

GA is a probabilistic and heuristic algorithm that uses a population of candidate solutions that evolve through a process of selection, crossover (recombination), and mutation. The initial population is generated randomly, and each individual is represented by a chromosome, which encodes the positions of the queens on the chessboard. The fitness function assigns a score to each individual based on how many queens are attacking each other. The time complexity of a genetic algorithm is $O(NGT)$ where N is the size of the population, G is the number of generations, and T is the number of evaluations (fitness function calculations) per individual. The space complexity is $O(N)$.

In terms of solving the 8-queens problem, Backtracking is a very efficient algorithm as it reduces the number of possibilities to explore, it is systematic, deterministic and the time complexity is lower. On the other hand, GA is a probabilistic method and the solution may not be the optimal one. It requires the definition of some parameters such as population size, mutation rate, and selection criteria which can affect the performance of the algorithm.

Example of Backtracking and Genetic Algorithm:

```
GA
Board size: 8
Time complexity: 0.05879998207092285
Backtrack
Board size: 8
Time complexity: 0.0009951591491699219
```

The output shows that backtracking is better than genetic algorithm [9]. Therefore, for small values of N, backtracking is an efficient method, while for large values of N, GA is a more suitable method.

Conclusion

In conclusion, the 8-Queens problem is a classic problem in computer science that can be solved using various methods such as Breadth-First Search (BFS), Depth-First Search (DFS), backtracking, and Genetic Algorithm (GA). Each method has its advantages and disadvantages in terms of time complexity, space complexity, and the quality of solutions. BFS is not a suitable approach for solving the 8-queens problem as it uses a queue data structure and has a high time and space complexity, $O(N! * N^2)$ and $O(N!)$ respectively. Similarly, DFS is also not an efficient approach for solving the 8-queens problem as it uses a stack data structure and has a high time and space complexity, $O(N! * N^2)$ and $O(N)$ respectively. Backtracking is a very efficient algorithm for solving the 8-queens problem as it has a lower time complexity, $O(N! * N^2)$, and space complexity $O(N^2)$. It is a systematic and deterministic algorithm that incrementally builds up a solution and uses backtracking to discard invalid solutions. GA is a probabilistic approach for solving the 8-queens problem. It uses a population of candidate solutions that evolve through a process of selection, crossover (recombination), and mutation. It has a time complexity of $O(NGT)$ where N is the size of the population, G is the number of generations, and T is the number of evaluations (fitness function calculations) per individual. The space complexity is $O(N)$. It generates a probabilistic solution but it requires defining some parameters such as population size, mutation rate, and selection criteria which can affect the performance of the algorithm.

References

- [1] "Eight queens puzzle," *Wikipedia*. Jan. 06, 2023. Accessed: Jan. 13, 2023. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Eight_queens_puzzle&oldid=1132028422
- [2] V. Keri, V. Kesri, and P. Ku. Pattnaik, "An Unique Solution for N queen Problem," *Int. J. Comput. Appl.*, vol. 43, no. 12, pp. 1–6, Apr. 2012, doi: 10.5120/6152-8539.
- [3] M. Eng, "Solving N-Queen Problem by DFS and BFS and Show Goal On Panel Control," *CodeProject*, Nov. 14, 2013. <https://www.codeproject.com/Articles/682129/Solving-N-Queen-Problem-by-DFS-and-BFS-and-Show-Go> (accessed Jan. 13, 2023).
- [4] "What is Big O Notation Explained: Space and Time Complexity," *freeCodeCamp.org*, Jan. 16, 2020. <https://www.freecodecamp.org/news/big-o-notation-why-it-matters-and-why-it-doesnt-1674cfa8a23c/> (accessed Sep. 24, 2022).

- [5] “Backtracking and N-Queen Problem,” *CodesDope*.
<https://www.codesdope.com/course/algorithms-backtracking/> (accessed Sep. 24, 2022).
- [6] J. Shankar, “N-Queens.” Sep. 26, 2022. Accessed: Jan. 13, 2023. [Online]. Available:
https://github.com/jaya-shankar/n_queens
- [7] A. Farhan and F. Awad, “Solving N Queen Problem using Genetic Algorithm,” *Int. J. Comput. Appl.*, vol. 122, pp. 975–8887, Jul. 2015.
- [8] S. Dhaka, “Backtracking and Genetic Algorithm Techniques.”
- [9] H. babayan, “GANQ.” May 29, 2020. Accessed: Sep. 26, 2022. [Online].
Available: <https://github.com/hiradbaba/GANQ>